Comp90042 Natural Language Processing

Workshop (Week 8 | 2026s1)

Dr Jean Lee

Agenda

1. Discussion (~20 mins)
   - **In-Context Learning**
   - **Instruction Tuning**

2. Programming (~35 mins)

27th April	8	workshop-08.pdf	11-instruction_sft_finetuning.ipynb	to be released on Friday evening

Recap: In-Context Learning

- Human can generally perform a new language task from only a few examples
  -> *"can LMs learn from a few-shot examples?"*
- **GPT-3 175B** : scaling up autoregressive LM -> emergent ability of **"in-context learning"**
- **In-context learning :** the model develops **a broad set of skills** at training time, and then **uses those abilities at inference time** to rapidly adapt to the desired task
  -> ***an ability that is non-existent in small models but rapidly improves performance in large models.***

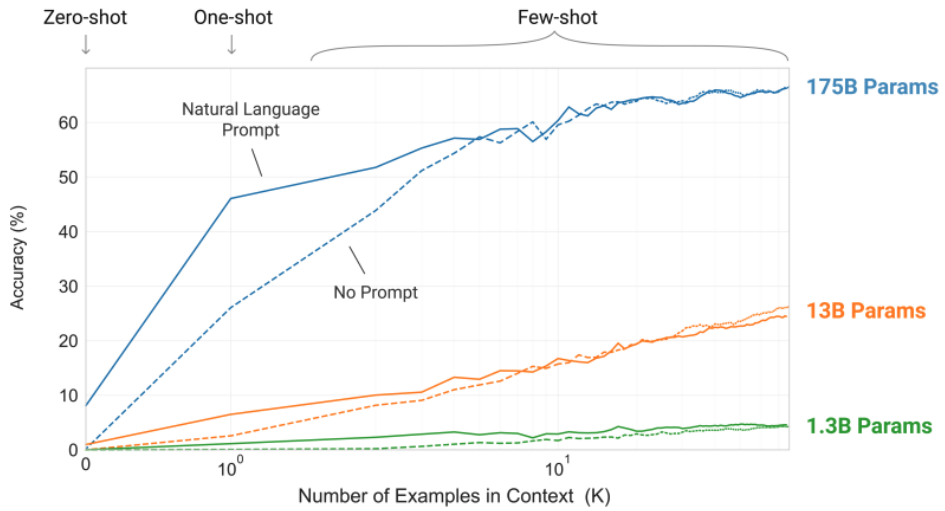


Fig : **Larger models make increasingly efficient use of in-context information.** The steeper "in-context learning curves" for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

No gradient updates

- **Zero-shot** : task description, prompt
- **One-shot** : + one example
- **Few-shot** : + a few examples

Gradient updates

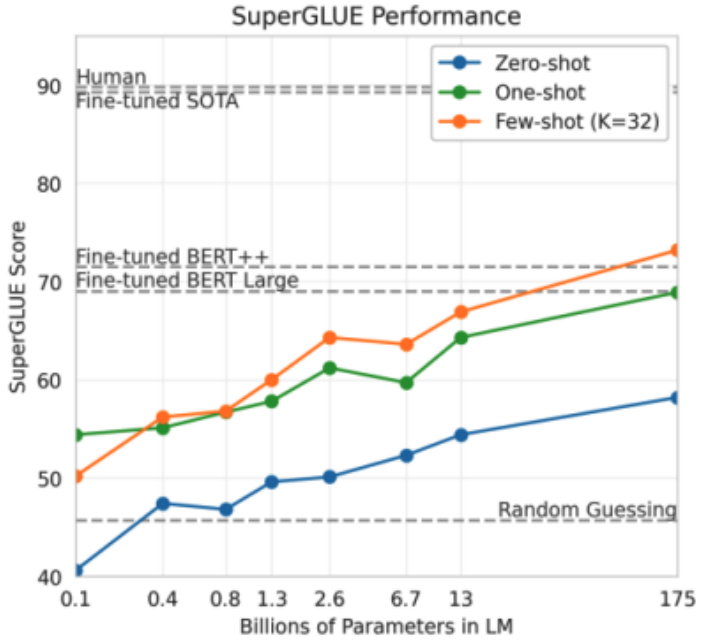
- **fine-tuning**

-[GPT 3] Language Models are Few-Shot Learners (TB Brown et al., OpenAI, May 2020, NIPS 2020) [paper]

Recap: In-Context Learning

GPT-3 (175B parameters)

- Specify a task by prepending examples of the tasks before your example
- Called 'in-context learning', to stress that no gradient updates are performed when learning a new task



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:          ← task description
2  sea otter => loutre de mer            ← examples
3  peppermint => menthe poivrée          ←
4  plush girafe => girafe peluche        ←
5  cheese =>                             ← prompt
```

In-Context Learning

Pros

- ***No Finetuning is needed.***
- *Prompt Engineering (e.g. Chain-of-Thought) can improve performance easily.*

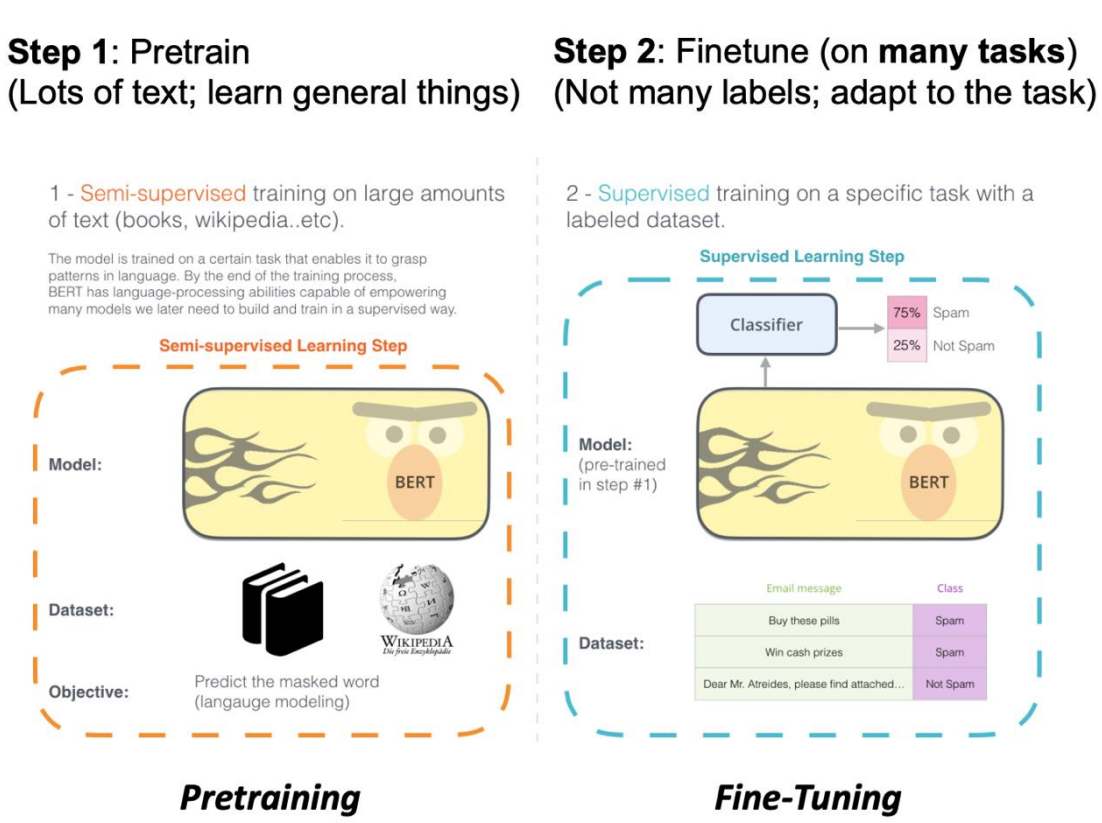
Cons

- *Limits to what you can fit in context.*
- *Complex tasks will probably need gradient steps.*

Ref.: Lecture 13-14 (p5, p38)

Recap: Fine-tuning

- Fine-tuning is an approach to transfer learning by **modifying the weights** of a pre-trained model to help it perform better on a **specific task or set of tasks**.
- **Full fine-tuning** : update all model parameters, use a smaller dataset



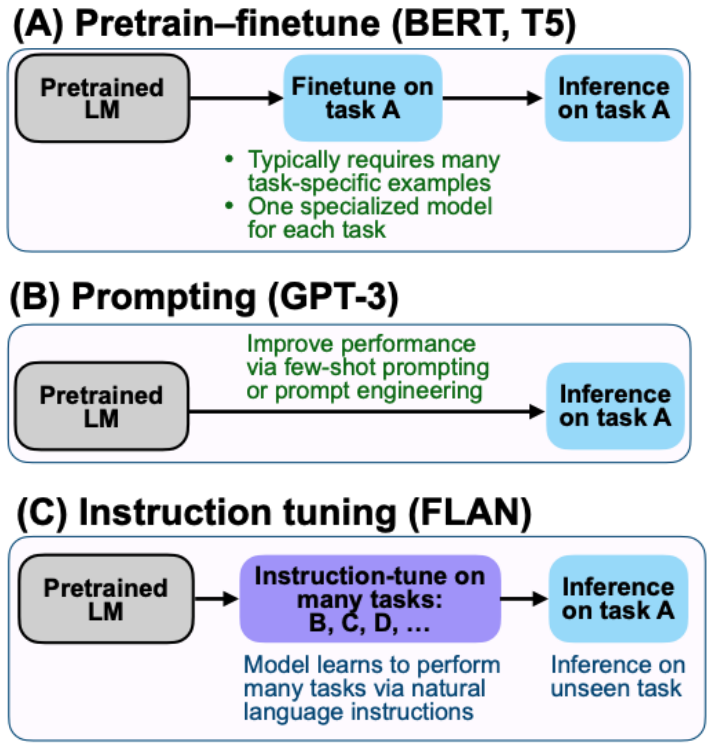


Fig 2: Comparing pretrain–finetune, prompting and instruction tuning

Ref.: Lecture 13-14 (p40, p51)

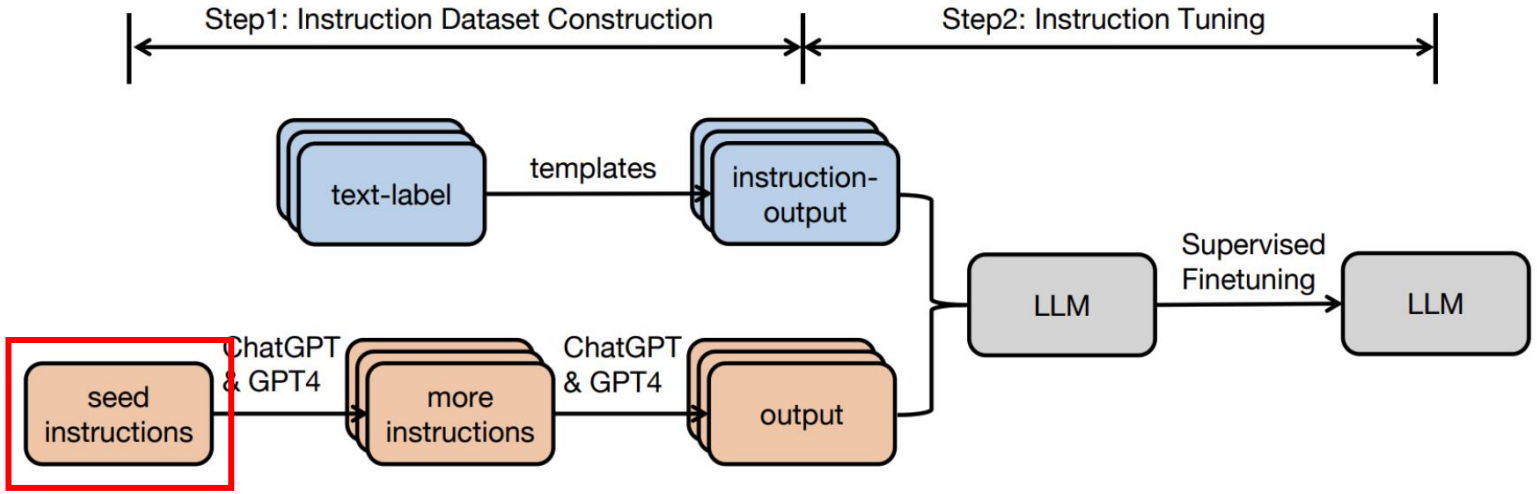
Fig2 : [FLAN] Finetuned Language Models Are Zero-Shot Learners (Google Research, ICLR 2021) [paper]

Recap: Instruction Tuning

What is Instruction Tuning?

- Post Training, Instruction Fine-Tuning, Supervised Fine-Tuning (SFT)
- Instruction Tuning = Supervised Fine-Tuning on **Instruction Data**

Instruction data (prompt, completion): (x, y)



(E.g. – classification tasks)

```
{"instruction":
"determine the sentiment of the
sentence.",
"instances":
[{"input": "He likes the cat.
 Is positive or negative?",
"output": "Positive."}],
"is_classification": true}
```

(E.g. – generation tasks)

```
{"id": "seed_task_12",
"name": "explain_behavior",
"instruction":
"Explain human's behavior.",
"instances":
[{"input": "Behavior: cry.",
"output": "There could be many
reasons why a person might cry.
They could be feeling sad,
scared, angry, or frustrated.
Sometimes people cry when they
are happy or relieved. There is
no one answer to why people
behave the way they do."}],
"is_classification": false}
```

Ref.: Lecture 13-14 (p41)

https://github.com/tatsu-lab/stanford_alpaca/blob/main/seed_tasks.jsonl

Recap: Instruction Tuning

Benefits and Limitation

Simple and straightforward, generalise to unseen tasks

The limitation of instruction finetuning:

1. It is **expensive to collect groundtruth data** for tasks
   Collecting demonstrations for so many tasks is expensive

2. There is **no right answer** for tasks like **open-ended creative** generation
   e.g. Write me a story about my dog and my son.

3. Even with instruction finetuning, there a mismatch between the Language Modelling objective and the objective of **"satisfy human preferences"**!

That's why we may consider RLHF (Reinforcement Learning from Human Feedback)

Ref.: Lecture 13-14 (p64)

Discussion:
In-Context Learning vs. Instruction Tuning

1. **Scenario:** A research team working on few-shot domain adaptation uses an LLM by providing 5 (five) manually constructed examples of protein structure classification tasks in the prompt. The model then predicts the class for a new protein without any parameter updates.

Answer: (a) ***In-Context Learning***

Justification:
In this case, the model is ***using examples provided at inference time*** *(few-shot examples) to guide its predictions.* ***No model parameters are updated****, making this in-line with in-context learning, where the model learns patterns dynamically* ***during inference from the prompt*** *itself* ***without fine-tuning****.*

Discussion:
In-Context Learning vs. Instruction Tuning

2. **Scenario:** To build a general-purpose assistant, developers fine-tune a pre-trained LLM on a dataset containing thousands of instruction-output pairs across 100+ tasks (e.g., summarisation, classification, translation), each framed with natural language directives.

Answer: (b) ***Instruction Tuning***

Justification:
This scenario explicitly describes the process of ***fine-tuning a model on a set of instruction-output pairs****. The model's* ***parameters are updated during training*** *to optimise performance on those tasks, making this a clear case of instruction tuning where the model adapts to follow various instructions consistently.*

Discussion: In-Context Learning vs. Instruction Tuning

3. **Scenario:** During evaluation, a model is prompted with: *"Classify the sentiment of this review: 'I loved the cinematography but the plot was boring.'"* The model correctly classifies it without having been trained specifically for sentiment analysis.

Answer: (a) ***In-Context Learning***

Justification:
The model correctly handles the sentiment classification task in ***a zero-shot manner, guided only by the prompt provided at inference time****. No training on sentiment-specific data is required, indicating that the model relies on its broad pretraining and in-context learning.*

Discussion:
In-Context Learning vs. Instruction Tuning

4. **Scenario:** A language model is trained with a supervised fine-tuning stage where each sample includes a task-specific instruction (e.g., *"Convert to passive voice"*) followed by a gold standard output. The resulting model can handle previously unseen instructions of similar format.

Answer: (b) ***Instruction Tuning***

Justification:
In this case, the model undergoes ***supervised fine-tuning****, which involves* ***updating its weights based on task-specific instructions.*** *This is a classic example of instruction tuning, where the model is explicitly trained to follow various natural language instructions more effectively.*

Programming!

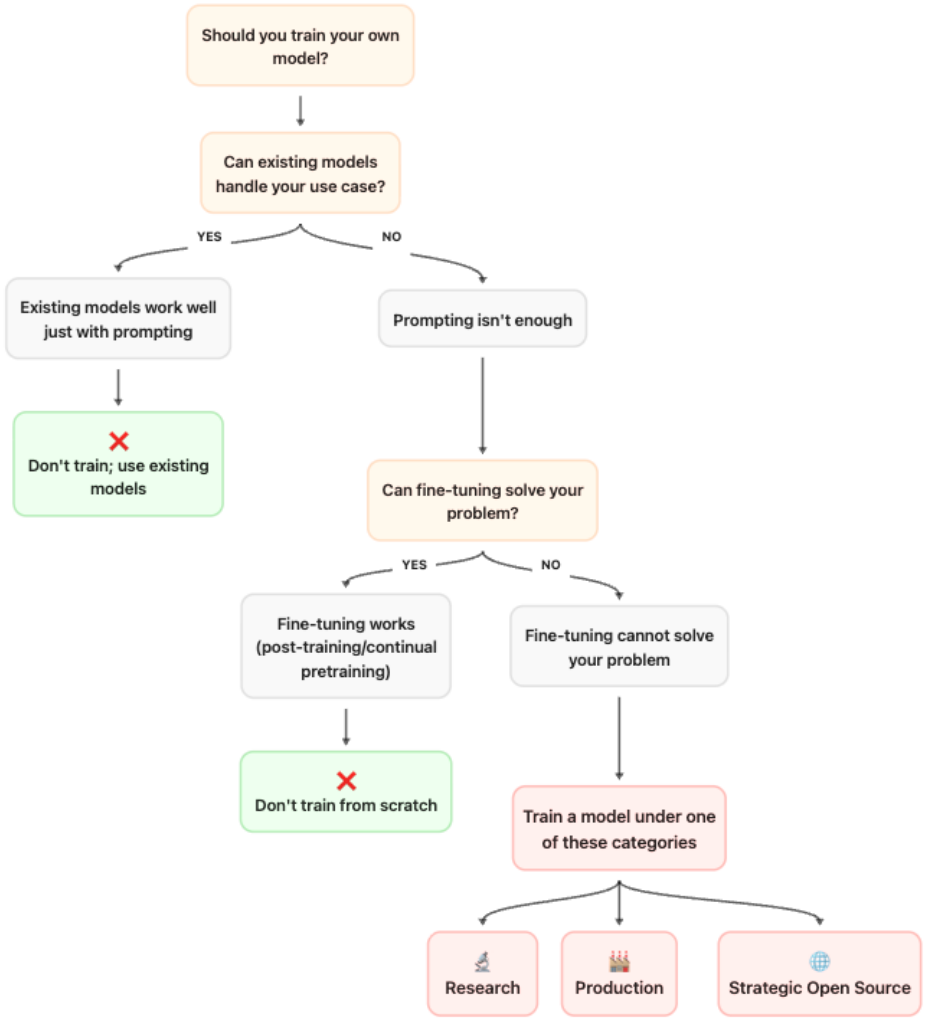
In this workshop, we will teach you how to 1) prepare your own instruction data, 2) conduct supervised fine-tuning, and 3) test your instruction fine-tuned model.

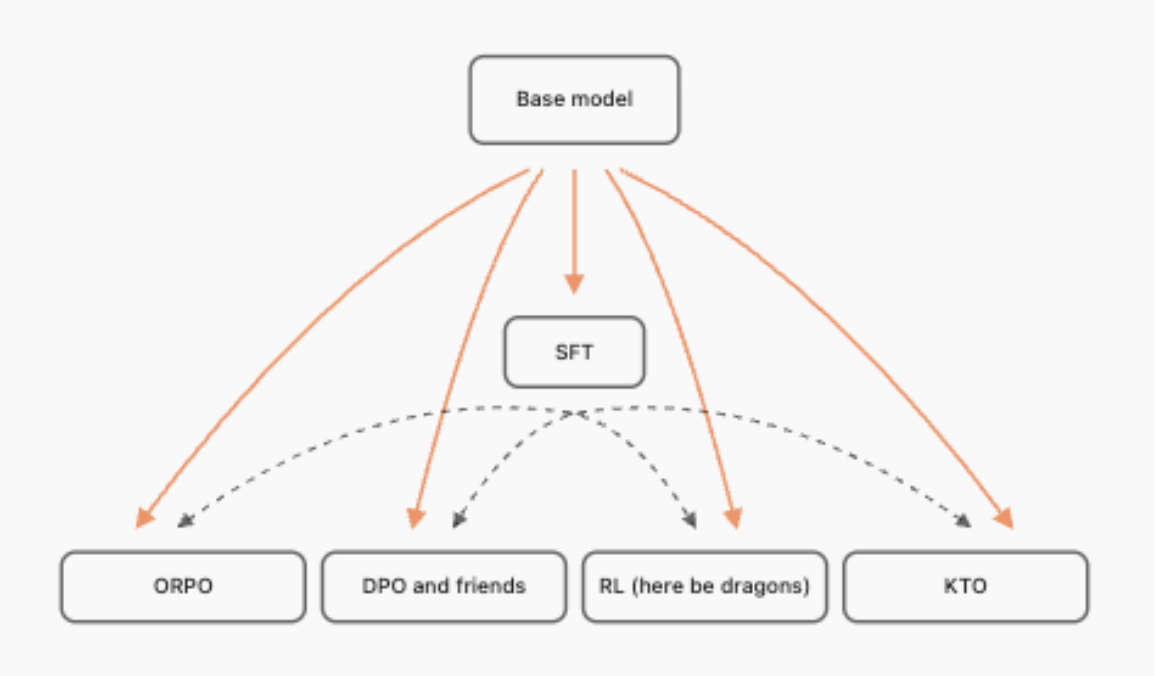
1. **Instruction Data Preparation**
   Provides a hands-on demonstration of how to prepare instruction data for fine-tuning conversational models by leveraging the transformers, trl, and datasets libraries.

2. **Supervised Fine Tuning**
   Demonstrates how to fine-tune the `HuggingFaceTB/SmolLM2-135M` model using the `SFTTrainer` from the `trl` library. The notebook cells run and will fine-tune the model.

3. **Test your fine-tuned model**
   Demonstrate how to test your model, which you fine-tuned in the previous steps, via your Huggingface Repository.

Programming! - The Smol Training Playbook





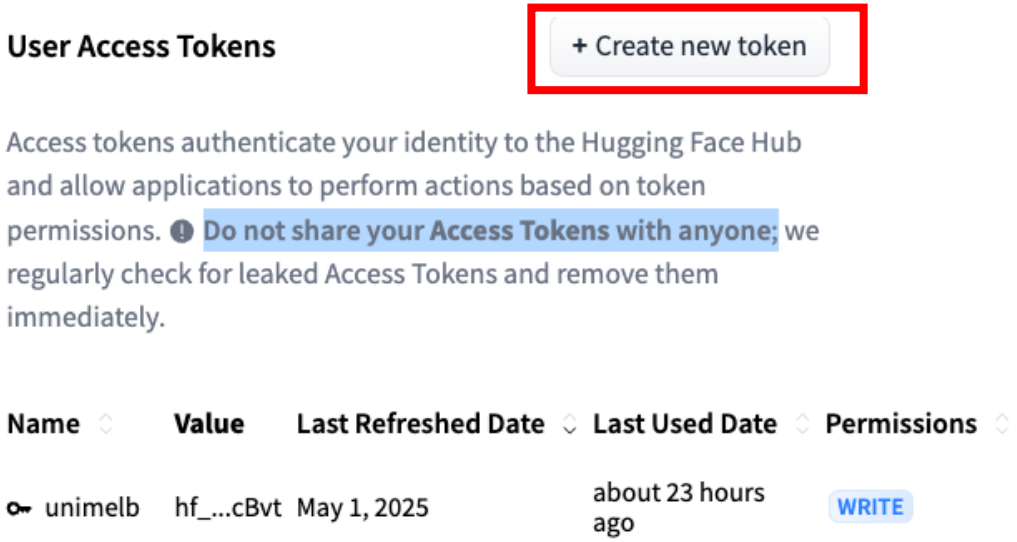
https://huggingface.co/spaces/HuggingFaceTB/smol-training-playbook

Programming!

Hugging Face API Keys to Google Colab

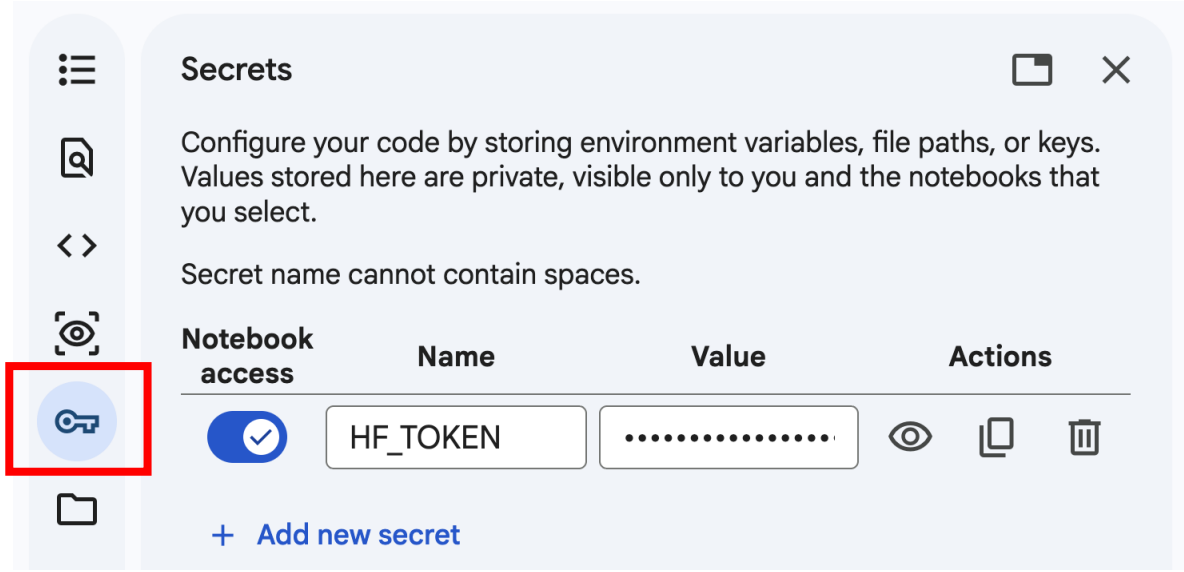
Hugging Face (HF) API Tokens

- ***Create an account on Hugging Face*: Go to Access Tokens [Link] → Create a new token (give it 'write' permission) → Copy and save your API key** *(if you forget it, that's okay—you can generate a new one).*



Google Colab to connect

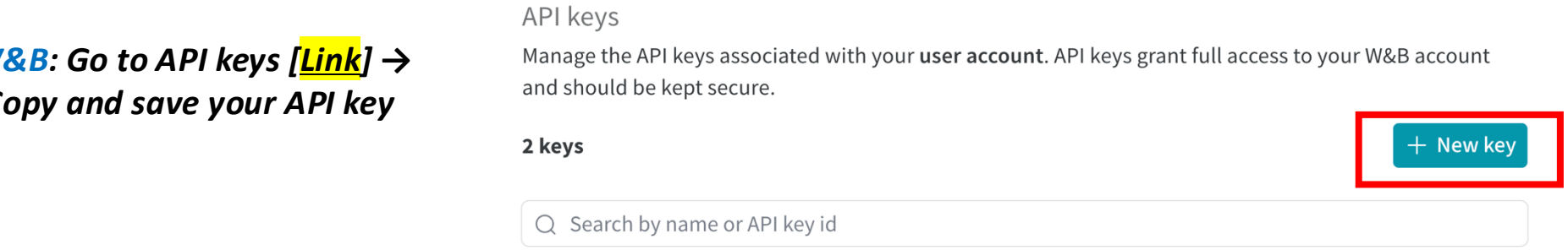
- ***In your Colab Notebook*: Click the 'key' icon on the left panel → Add a new secret → Allow notebook access → Name it 'HF_TOKEN' → Paste your Hugging Face API key as the value.**



Programming!

W&B API Keys to Google Colab

- ***Create an account on W&B**: Go to API keys [Link] → Create a new token → Copy and save your API key*



- ***In your Colab Notebook**: Run the code block → Follow the instruction*

```
1 import wandb
2
3 wandb.login()
```

```
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter: ··········
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: jeanlee to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
True
```